Realtime Fallacy Detection

Sahil Mangotra
Department of Computer Science
Kennesaw State University
Kennesaw, GA
smangotr@students.kennesaw.edu

Robel Mamo
Department of Computer Science
Kennesaw State University
Kennesaw, GA
rmamo@students.kennesaw.edu

John Yare
Department of Electrical and Computer Engineering
Kennesaw State University
Kennesaw, GA
jyare1@students.kennesaw.edu

Abstract*—In today's digital landscape, social media platforms frequently reinforce users' existing beliefs, creating echo chambers where flawed reasoning spreads unchecked. These environments, combined with reduced attention spans, make it increasingly difficult for individuals to critically evaluate the arguments they encounter. To address this challenge, we developed an automated system for real-time detection of logical fallacies in both text and spoken arguments. Our system follows a three-stage pipeline: (1) audio input is transcribed using a CNN encoder with a BiLSTM decoder, (2) the resulting text is cleaned and normalized, and (3) a fallacy classifier assigns one of several fallacy labels. We experimented with classical machine-learning approaches—including Naïve Bayes, Logistic Regression, and Support Vector Machines—using TF-IDF features. These baseline models achieved moderate performance (maximum accuracy ~60%) but were limited by their inability to capture semantic context. To overcome this, we shifted to Transformer-based architecture. RoBERTa provided an initial improvement, and fine-tuning Microsoft DeBERTa v3 further enhanced contextual reasoning. The major breakthrough came from addressing extreme class imbalance by applying a custom weighted loss function. This improved overall classification performance to 71.2% accuracy and 71.8% weighted F1-score, while significantly reducing false positives—particularly in moderator speech during real-world debate tests. For the audio pipeline, our speech-to-text model achieved strong transcription performance with a Character Error Rate of 0.097 (≈90% accuracy) and a Word Error Rate of ~0.30. Overall, our results demonstrate that combining robust transcription with context-aware Transformer models and weighted training strategies yields a viable real-time fallacy detection system. This tool has the potential to enhance digital literacy by helping users recognize manipulative or logically flawed arguments as they encounter them. As a result, we aim to develop an end-to-end real-time fallacy detection system, with the optimal speed to accuracy trade-off.

Keywords—NLP, Machine Learning, State Vector Machines, Support Vector Machines, Deep Learning, CNN, LSTM

I. Introduction

The modern digital world has an exponential increase in the distribution of information, particularly on social media. One consequence of this environment is presence of "echo chambers", which can amplify the information which confirm existing beliefs, regardless of soundness of the argument involved. This coupled with decreasing attention spans, lowers the chances that users will carefully evaluate the information presented to them. Thus, automatically detecting fallacy, flawed reasoning, misleading, and propagandistic reasoning is a key resource for promoting digital literacy and fighting misinformation.

Even though human ability to detect logical fallacy is well documented, automatically detecting logical fallacies is a significant Natural Language Processing (NLP) challenge. It requires nuanced language understanding, context and the logical relationship between premise and conclusion. For example, distinguishing a casual criticism from and ad hominem attack or a causal relationship from potentially false causality requires a model to leverage information beyond surface level or overt features.

Our project tackles this problem by proposing a clear goal, to design, implement and compare models for multi-class classification of logical fallacies. To solve this problem, first we have to fix a major problem of scarcity of data. We create a new, large scale and diverse dataset by merging LOGIC dataset with real world SemEval-2020 Propaganda Detection Corpus made up of systematic parsing and merging. This involves a label mapping process that combines the different terminologies of two different dataset fields. In addition, the merged dataset had an extremely imbalanced class distribution mirroring the real world, where some fallacies are much more prevalent than others, which adds a key technical issue because a naive model can achieve high accuracy by ignoring the rare classes all together.

The primary contribution of this study is the development of this unique, combined dataset and an extensive baseline evaluation of classical machine learning algorithms. We train and compare Naïve Bays, Logistic Regression, and Support Vector Machine (SVM) using TF-IDF features. Our result establishes a performance baseline, with the SVM achieving a weighted F1-score of 42.2%. We also provide a confusion matrix for a detailed error analysis, explaining specifically what that baseline identifies well, where it struggles, which fallacies it can distinguish easily, and which are still very confusable afforded its baseline approach. This report describes our process, experimental setup, and results of this baseline comparison.

Prior work has explored logical fallacy detection primarily in text-only settings. Jin et al. introduce the LOGIC and LOGIC-Climate datasets which show that fine-tuned transformer models like Electra and RoBERTa can achieve micro-F1 scores at ~50%

range and they struggle with rare fallacy types due to label imbalance and some subtle linguistic cues [1]. The NLP4IF shared task and SemEval-2020 Task 11 extends this direction to fine-grained propaganda techniques for propaganda detection where even the best systems obtain relatively lower fragment-level F1 because the models must localize both the span and the specific technique [2][3]. More recent work on explainable models and structure-aware classifiers further highlights the importance of modelling structure and providing interpretable signals for human analysis [5][6].

Our project extends these ideas to a multimodal, real-time setting in which fallacies are detected from speech as it is produced. We mainly focus on three main objectives:

1. To construct a unified fallacy dataset that combines LOGIC with propaganda fragments and negative examples from news articles,
2. To benchmark classical models and transformer-based classifiers on this dataset to really understand the trade-off between simplicity and performance, and
3. To integrate the best-performing model into an end-to-end pipeline that converts audio to text, predicts fallacy labels at the snippet levels, and aggregate them into a speaker-level scorecard.

II. RELATED WORK

The automated identification of logical fallacies is an emerging area of research in Natural Language Processing that is related to the more well-established areas of argumentation mining and propaganda technique identification. The research can be divided into two high-level categories based upon the datasets utilized and the complexity of models ranging from classical machine learning baselines to cutting-edge transformer-based architectures.

A. Foundational Datasets and Task Definition

The absence of large-scale, high-quality, labeled data has been a major obstacle for detection of fallacies. Jin et al. (2022) helped to address this issue with the release of the LOGIC dataset, which is an important resource for our project [1]. Jin et al. then benchmarked several current models including existing pre-trained language models, such as BERT and others, and determined that large, existing models performed poorly on this task [1]. This work underscores that the effort to detect a logical fallacy is a very complicated reasoning activity that even high-performance, standard models are struggling with, and they suggest better datasets or more specialized model architectures [1]. At the same time, Da San Martino et al. (2019, 2020) developed the Propaganda Techniques Corpus (PTC) during the NLP4IF-2019 workshop [2] and later as part of SemEval-2020 Task 11 [3]. They defined 18 (later 14) propaganda techniques and produced a large, fragment-level annotated corpus [3]. Ultimately, those datasets were treated as having a completely different task as it related to fallacies despite the considerable parallel conceptual overlap with named labels (e.g. Name Calling, Labeling and ad hominem).

B. Classical Machine Learning Approaches

Classic and early contributions to text classification created methods that are important baselines for this task. Joachims (1998) offered a rationale for our baseline by examining the performance of Support Vector Machines (SVMs) in text classification [4]. This work showed that SVMs are particularly suitable for the extraction of information from high-dimensional, sparse data, which is the exact form that the TF-IDF vectorization produces [4]. Ermurachi et al. (2020) used this classical approach to the SemEval-2020 propaganda dataset by using traditional TF-IDF and n-gram features with models like Logistic Regression [5]. The system produced a micro-average F1-score of 0.43 for the technique classification task. However, their results did not come close to the performance of the winning transformer models, thereby providing evidence for the ceiling performance of classical, non-contextual approaches [5].

C. State-of-the-Art Transformer Approaches

Currently, fine-tuned transformer models are state-of-the-art for both propaganda and fallacy detection. The SemEval-2020 Task 11 competition featured many successful systems built on this architecture. For example, Altiti et al. (2020) participated in the Technique Classification subtask using a BERT uncased pre-trained language model [6]. Their model resulted in an F1-Score of 0.553, outperforming the baseline provided by the task organizers [6]. Kranzlein et al. (2020) developed neural models for the same task and used a combination of finetuning, data augmentation, and feature engineering to improve performance [7].

While these systems demonstrated that transformers are effective, more recent work has also illuminated the limitations of only using simple fine-tuning in the context of more complex reasoning. For example, Sourati et al. (2022) argue that using the language model itself, in the absence of additional objective guidance, is " not sufficient enough to fully solve the task" [10]. They introduce a more rigorous three-stage framework for fallacy identification and explore individual methods for explainability [10]. A shared focus on deeper reasoning is also seen in Jeong et al. (2025), who found that Large Language Models become "Better Logical Fallacy Reasoners," when the model is prompted with counterarguments, explanations, and goals, instead of just simple classification [8]. Other work is looking for augmenting transformers with explicit structural information. For example, Lei and Huang (2024) "Boosting Logical Fallacy Reasoning" by giving a RoBERTa-model-comparison a "logical structure tree" [11]. In the case of Lalwani et al. (2024), a more explicit resolution to the prior work done by Sourati et al. is bypassing all traditional classification entirely by using the LLM to translate natural

language into First-Order Logic (FOL) and then using SMT solvers to validate the structure of the argument, effectively revising how we are considering and moving towards the neuro-symbolic [9].

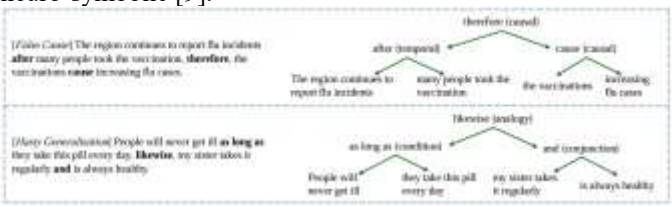

Figure 1: Examples of logical fallacies and their logical structure of trees [11].

Figure 1 illustrates how recent work uses logical structure trees to encode augmentative relations [11]. Our current system instead operates directly over token sequences but could integrate this kind of structures signals in future work.

III. Methodology

Our real-time fallacy detection system consists of two primary components: a speech recognition module and a fallacy detection/classification module working in conjunction.

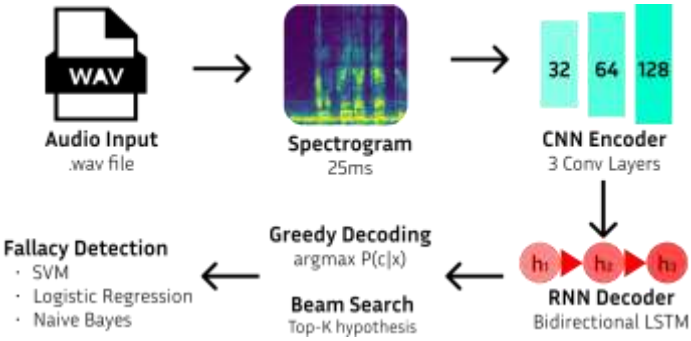

Figure 2: Proposed audio to fallacy detection pipeline

Speech Recognition Module: The speech-to-text conversion employs a hybrid deep learning architecture combining convolutional and recurrent neural networks for end-to-end speech transcription. The inputs, in the form of wav files, are preprocessed into log-mel spectrograms capturing relevant acoustic features. The spectrograms are processed by CNN encoder with three convolutional layers with 32, 64, and 128 filters respectively, followed by batch normalization and max pooling layers to extract robust features. The output encoded feature vectors are then fed into a two-layer bidirectional LSTM decoder. Character-level probability distributions over our 29-character vocabulary (i.e., the alphabet, space character, apostrophe, and a <blank> token) are predicted at the output of the decoder. All alphabet characters are lowercased. For the character to word decoding process, we plan to compare two decoding strategies: greedy decoding, and beam search.

Text Preprocessing and Features Extraction: We normalize each transcript span so that models learn rhetorical patterns instead of superficial casing and punctuation before classification. We apply lowercasing, tokenization, and lemmatization by doing lowercase on all text, then tokenize using a regex that keeps word-like tokens including internal apostrophes in words like "don't", and then lemmatize each token with WordNetLemmatizer to collapse inflected forms to a shared base form ("attacking," "attacked," and "attacks" all become "attack"). We also remove rows that are just links, drop rows with missing text, and filter out the catchall label "miscellaneous," leaving 9,727 labeled spans across our mapped fallacy classes. We then featurize text using `TfidfVectorizer` with our custom analyzer. This produces TF-IDF-weighted features over the lemmatized tokens. Those features feed the Multinomial Naive Bayes, Logistic Regression, and a Support Vector Machine (SVM) with probability estimates enabled. These classical models act as fast, lightweight baselines for fallacy detection and they let us quantify how far we get with surface-level lexical cues before moving to transformer-based models.

Fallacy Classification Module: The transcribed text output from the speech recognition module is fed directly into a supervised learning classifier for real-time fallacy detection. We implemented and compared three classification approaches: Naive Bayes, Logistic Regression, and Support Vector Machines (SVM). The models are trained on a combined dataset consisting of the LOGIC dataset, which provides diverse examples of logical fallacies across multiple categories, and the Propaganda Detection dataset, which offers additional examples of manipulative reasoning patterns. This multi-source training strategy is designed to improve model robustness and generalization across different fallacy types and rhetorical contexts. We evaluate classifier performance using standard multi-class classification metrics including overall accuracy, weighted precision, weighted recall, and weighted F1-score, with the weighted variants accounting for potential class imbalance in fallacy type distributions.

IV. Experimental setup

We ran our experiments in Python 3, mainly with Google Colab to foster collaborative development in a consistent environment. Key libraries which we leveraged included pandas for working with and examining data, and scikit-learn to implement TF-IDF vectorization and baseline classifiers: Multinomial Naive Bayes, Logistic Regression (max_iter=1000), and a Support Vector Machine (SVM) with a linear kernel. The baseline experiments would be efficient even in traditional cloud environments, but we will be running additional experiments which consume larger amounts of compute resources, such as fine-tuning transformer models like RoBERTa or training of Audio-to-text models, on local hardware (in this case, Intel i7 and NVIDIA RTX-series 2070 Ti) for quicker training. We used a PyTorch-based stack with HuggingFace transformers for the transformer and speech

components of our system. The requirements file specifically lists PyTorch 2.6.0 with the CUDA 12.4 build and transformer 4.49.0 along with other supporting libraries like pandas, scikit-learn, etc.

A. Dataset Description

As stated in the introduction, our research utilizes a novel, combined dataset specifically curated for the task of multi-class logical fallacy detection. This dataset integrates two primary sources: the LOGIC dataset and the Propaganda Techniques Corpus (PTC) from SemEval-2020 Task 11.

1. LOGIC Dataset

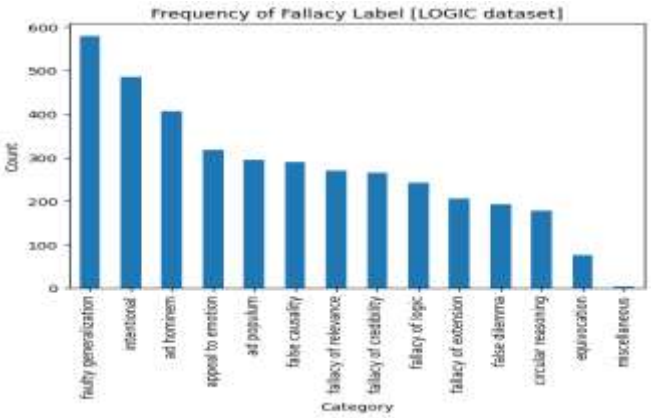

Figure 3: Frequency distribution of LOGIC dataset

The LOGIC dataset gives the basis for this task, which was designed and annotated by experts in various text sources of approximately 3,800 instances and 13 types of logical fallacies. The bar chart in figure 3 presents frequency distributions over the labels in this set. Faulty generalization, intentionally, and ad hominem are the most frequently occurring classes, while miscellaneous and equivocation had the least.

2. Propaganda Techniques Corpus (PTC)

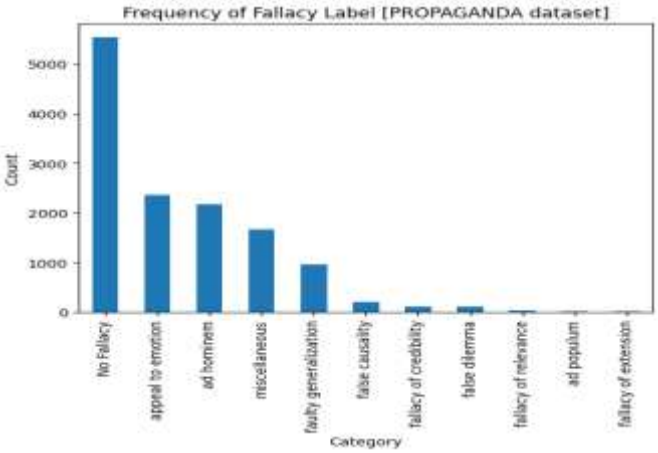

Figure 4: Frequency distribution of PTC dataset

The Propaganda Techniques Corpus (PTC), described by Da San Martino et al., was developed for the SemEval-2020 competition and consists of approximately 550 news articles. The training set comprises 371 articles. We utilized the fragment-level (FLC) annotations to extract labeled text fragments.

Our contribution involved developing a parsing pipeline to process the PTC. This pipeline extracted the labeled text fragments using the provided offsets and also identified unlabeled text segments between labeled spans, assigning these a new label, "No Fallacy". We then applied a systematic label mapping (detailed in the Methodology section) to translate the 18 original propaganda techniques into our target logical fallacy categories, handling multi-label spans by creating duplicate entries. After this processing, the derived propaganda dataset contained 13,161 instances. Figure 4 shows the frequency distribution after mapping. "No Fallacy" is by far the dominant class, followed by appeal to emotion, ad hominem, and miscellaneous.

3. Final Fallacy Detection Dataset

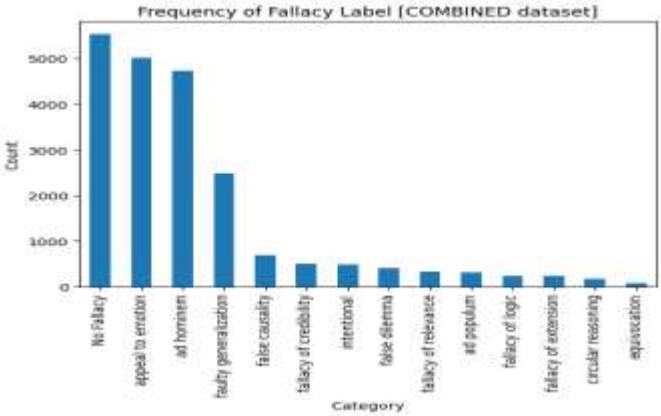

Figure 5: Frequency Distribution of Final dataset

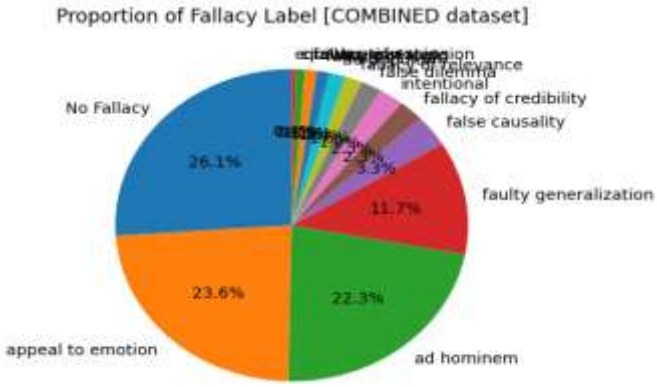

Figure 6: Label proportions of Final dataset

The final combined dataset brings together the processed data from both the final LOGIC and mapped PTC dataset. The combined dataset consists of 21,227 cases once cleaned (removing missing values and possible URLs) from both sources. It was split at an 80/20 split to form a training set (~16,981 cases) to use in developing our model, and a test set (~4,246 cases) to evaluate our model. As seen in Figures 5 & 6, the label distribution of the final combined data was heavily imbalanced with the most frequent label of "no fallacy" being in 26.1% of the total dataset [See Figure 6]. The next most prevalent classes in the final combined data were an appeal to emotion (23.61%) and ad hominem (22.26%) largely due to the commonality of these fallacy types in the propaganda data of the PTC. Several fallacy types such as equivocation, circular reasoning, and fallacy of extension are still severely underrepresented in the final combined dataset being represented in less than 1% of the total cases [See Figure 6]. Table 1 contains some important statistics for the cases included in the final dataset.

Statistic	**Value**
Total Instances	21,227
Number of Unique Classes	14
Training Instances (80%)	~16,981
Testing Instances (20%)	~4,246
Most Frequent (No Fallacy)	~26.1%
Least Frequent (Equivocation)	~0.4%

Table 1

4. LibriSpeech Dataset

The LibriSpeech dataset is an audio-to-speech dataset that is composed of 981.2 hours of 16kHz English audio speech and corresponding text data. The data is derived from audiobooks read from LibriVox project and has been carefully segmented and aligned. For training, the dataset is split into 3: train-clean-100 (100 hours), train-clean-360 (360 hours), and train-other-500 (500 hours). For testing, it is split into 2: dev-clean (5.4 hours) and dev-other (5.3 hours). Finally, for testing, it is again split into 2: test-clean (5.4 hours) and test-other (5.1 hours). Furthermore, all these files combined contain 2,484 speakers, providing diverse speech data. Although the files are in flac file format, they will be converted to wav file before the preprocessing stage. We aim to split each of the audio files into chunks of 25 milliseconds audio files and their corresponding text files.

Set	**Hours**
Train	960
Test	10.7
Validation	10.5
Total	**981.2**

Table 2: Total number of hours for each set

In our experiment we used the train-clean-100 with 100 hours of labeled audio. We created an 80-20% split corresponding to 22,832 training and 5,708 testing samples, respectively. Our pipeline included converting a text file to a csv splitting key attributes with a delimiter. Then, extracting audio file names and their respective transcription using simple regex definitions as follows respectively: **r'[\d+\-]'** and **r'([a-z]+ {0,})'**.

B. Evaluation Metrics

Fallacy Classification Model Evaluation: We report four main metrics for multi-class fallacy classification: overall accuracy, weighted precision, weighted recall, and weighted F1-score. Accuracy is the fraction of the test examples where the predicted label matches the true label. Weighted precision and weighted recall average per-class precision and recall while weighting each class by it supports, i.e, its number of examples. Weighted F1-score is the corresponding weighted harmonic mean of precision and recall. We use the weighted versions because our dataset is not balanced. For example, after merging LOGIC and the processed PTC, “No Fallacy” alone accounts for about 26% of all labeled spans, while categories like equivocation, circular reasoning, and fallacy of extension each make up < 1% of the total.
We also keep the full classes of precision, recall, and F1 table in the classification report to see which fallacy types are being detected reliably and which ones are confused. This is very important because some fallacy types like “ad hominem” and “appeal to emotion” are frequent and well-known, while others appear in only a handful of examples.

Speech-To-Text Model Evaluation: For the speech-to-text model, we generate character-level predictions across 50 sequential timesteps, capturing fine-grained temporal dependencies within the audio input. After obtaining the full sequence of predicted characters, we perform a greedy decoding step to reconstruct words, assembling the most likely character outputs without beam search or additional language-model constraints. To assess the performance of this pipeline, the model was evaluated using both character error rate **(CER)** and word error rate **(WER)**, providing insight into its accuracy at the sub word level as well as its effectiveness in producing coherent, correctly spelled words.

C. Implementation

Fallacy Classification Model: For combining the 2 datasets, we perform labeling to map the 20 propaganda technique labels in the Propaganda dataset to their corresponding logical fallacy labels from the LOGIC dataset. This is implemented using a simple dictionary/hash map for fast lookup. We then use panda dataframes to concatenate the 2 datasets together and split into 80-20 splits for training and testing. Three classical machine learning models are implemented for the fallacy classification

task. Each of the models uses TF-IDF to convert the text input into feature vectors. The model with the highest F1-score is selected as the best performer. The selected model is then used to generate a normalized confusion matrix to analyze per-class performance.

Speech-To-Text Model: The speech-to-text model is made of two components: a CNN encoder and an RNN decoder. The encoder is made of 3 sequential blocks, with each of the sequential blocks containing a convolutional block, with 32, 64, 128 feature outputs respectively, followed by batch normalization and a ReLU activation. The first 2 sequential blocks are also followed by a 2x2 maxpooling layer to divide the size of the image by 2 while multiplying the number of channels by the same scale we are dividing the image size. The output of the final sequential block is followed by a flattening layer and a compacting it to a (256, 1) vector. The RNN decoder is a BiLSTM with 2 sequential blocks, followed by a dropout set at 30%. The first layer has an input of 256 features and a concatenated output of 512, and the second layer has an input of 512 features and an output of 512. The output is flattened into a (29, 1) vector, representing the probability of each character. For the loss function, we aim to use CTC loss, a commonly used loss function for speech-to-text model training.

D. Results and Discussion

Fallacy Classification Model Results: We conducted an initial evaluation of the three classifiers using preprocessed and vectorized text. The results are summarized in table 3 below:

Model	Accuracy	Weighted Precision	Weighted Recall	Weighted F1-score
Naive Bayes	0.52	0.41	0.52	0.43
Logistic Regression	0.57	0.57	0.57	0.53
SVM	0.60	0.60	0.60	0.58

Table 3: Final Model Comparison

Among the baseline models, the Support Vector Machine (SVM) achieved the highest weighted F1-score of **0.58**, making it the strongest traditional classifier. The confusion matrix (shown in Figure 7) shows that SVM performs reliably on high-frequency fallacies such as *ad hominem* and *appeal to emotion* but struggles with rare categories like *equivocation* and *circular reasoning*. These trends were expected due to significant class imbalance across the combined fallacy datasets. Overall, the baseline results validate the effectiveness of our preprocessing pipeline; tokenization, lowercasing, and lemmatization, and the use of TF-IDF features for initial experimentation.

To address the limitations of these statistical models, we next transitioned to Transformer-based architectures capable of modeling deeper semantic and contextual relationships. Our initial model, RoBERTa-base, provided moderate improvements, increasing overall accuracy to approximately 64%. This demonstrated the advantage of contextual embeddings, particularly in distinguishing subtle linguistic differences between non-fallacious emotional language and actual *appeal to emotion* fallacies. However, RoBERTa still struggled with the dataset's extreme class imbalance. Like the baseline models, it disproportionately predicted majority-class fallacies and rarely identified low-frequency errors, such as *equivocation* or *fallacy of extension*.

To further improve contextual reasoning, we adopted Microsoft's DeBERTa v3, which leverages disentangled attention and enhanced mask decoding. DeBERTa better captured argumentative structure and reduced confusion between conceptually similar fallacies (e.g., *false dilemma* vs. *appeal to emotion*). Despite these architectural benefits, the model continued to favor frequent classes until we introduced a **custom weighted loss function**. By assigning higher penalties to misclassifications of rare fallacies, the weighted loss rebalanced the learning process and forced the model to treat all fallacy types—common or rare—as equally important. This modification led to a substantial performance improvement. The final weighted DeBERTa model achieved **71.2% accuracy** and a **71.8% weighted F1-score**, significantly outperforming all baseline and unweighted Transformer models. Additionally, the model demonstrated improved recall for rare fallacies and reduced false positives on non-fallacious statements, correcting several major bias issues observed in earlier stages of development.

Overall, these results show that while traditional classifiers provide a reasonable foundation, Transformer-based models, especially when combined with weighted loss training, deliver superior performance in fallacy detection. Their ability to capture contextual nuances and recognize less frequent fallacy types makes them far more suitable for real-world argument analysis.

We compared our results for contextualization with reported performance on standard logical fallacy and propaganda benchmarks. Jin et al. report that fine-tuned Electra and DeBERTa models achieve micro-F1 scores around 53.31% and 50.29% on the original LOGIC dataset while their structure-aware Electra-StructAware model improves the overall F1 to 58.77% [1]. Lalwani et al. translate natural language arguments to first-order logic in their NL2FOL model and it reasons with SMT solvers to achieve an F1-score of 71% on LOGIC showing the benefit of good logical structure [9]. The NLP4IF shared task and SemEval-2020 task 11 report relatively lower scores for fine-grained propaganda detection with the best systems reaching F1 ~24-25% ranges [2][3], and a BERT-based system from JUST obtaining a macro F1 of ~55.31% on technique classification [6]. Our final DeBERTa classifier on the merged LOGIC + propaganda dataset achieves 71.2% accuracy and 71.8% weighted F1, which is competitive with the best reported LOGIC results even though our label space and training data differs. This suggests that combining a large and heterogenous dataset with a strong transformer

encoder allows our model to close most of the gaps between traditional supervised baselines and more complex structure-aware approaches.

Model & Dataset	Metric	Score	Source
Electra (LOGIC)	Micro-F1 score	53.31%	[1]
DeBERTa (LOGIC)	Micro-F1 score	50.29%	[1]
Electra-StructAware (LOGIC)	Micro-F1 score	58.77%	[1]
NL2FOL (LOGIC)	F1 score	71.0%	[9]
Best SemEval-2020 propaganda systems	F1 score	~24-25%	[2][3]
JUST BERT (SemEval-2020 Task 11)	Macro-F1 score	55.31%	[6]
Our Weighted DeBERTa (combined dataset)	Weighted F1 score	71.8%	N/A

Table 4: Comparison of all reported results on LOGIC & propaganda datasets.

We also examined the per-class behavior of our strongest classical baseline using the SVM classification report and confusion matrix. Frequent classes like "No Fallacy" (F1=62%), "Ad Hominem" (F1=86%), "Appeal to Emotion" (F1=71%), and "Faulty Generalization" (F1=46%) are detected more reliably, reflected by the larger number of training examples available for these types. In contrast, rare categories like "Equivocation" (F1=0%), "Intentional" (F1=0.02%) are often misclassified as more common labels or as "no fallacy". This pattern mirrors the label distribution observed in our dataset analysis and highlights the need for class-balancing strategies, data augmentation, or specialized loss functions in the transformer-based models to improve recall on minority fallacy types.

Figure 7: Normalized confusion matrix for the best TF-IDF baseline i.e, SVM (Support Vector Machine)

The matrix in Figure 7 above shows strong diagonal mass for frequent classes like No Fallacy, Ad Hominem, and Appeal to Emotion. It also shows heavy off-diagonal confusion for rare classes like equivocation and fallacy of relevance, illustrating the impact of class imbalance on the model's performance.

Speech-To-Text Model Results: The speech-to-text model demonstrated strong performance at the character level, achieving a Character Error Rate (CER) of 0.097, corresponding to a character-level accuracy of approximately 90.23%. This indicates that the model is generally effective at capturing fine-grained phonetic details and producing correct character sequences. However, performance decreased at the word level, with a Word Error Rate (WER) of roughly 0.30, or about 70% word-level accuracy. The discrepancy between CER and WER suggests that while individual character predictions are often correct, accumulated errors—such as incorrect character insertions, deletions, or substitutions—can disrupt full word formation. This highlights the speech-to-text model's difficulty in consistently generating coherent, fully accurate word outputs despite strong sub-word accuracy.

In summary, our system demonstrates that real-time fallacy detection is feasible through a hybrid pipeline combining robust speech recognition with a weighted transformer classifier. Our key achievements include a 71.8% weighted F1-score on a merged, imbalanced dataset and improved recall for rare fallacies. However, some limitations are present; several fallacy types are still underrepresented, speech recognition errors can easily blur fallacy boundaries, and the system does not yet handle overlapping speech or complex multi-sentence reasoning. Future work should focus on expanding the dataset for rare fallacies, integrating logical structure trees for better reasoning, and tightening the coupling between ASR (Automatic Speech Recognition) and NLP (Natural Language Processing) components to reduce transcription-induced errors.

V. Conclusion

We have presented the design and initial implementation of a system for real-time detection in spoken arguments. This project addresses a critical need for tools that can promote digital literacy by automatically identifying flawed reasoning in environments like social media and public discourse.

Our work till date has established a complete pipeline, from audio transcription to fallacy classification. We have also successfully created a novel and combined dataset by integrating the LOGIC dataset with the Propaganda Techniques Corpus (PTC) addressing the initial challenge of data scarcity for this task. This dataset reflects the real-world distribution of fallacy types.

We implemented and evaluated three classical machine learning models, namely Naive Bayes, Logistic Regression, and Support Vector Machine (SVM), all using a TF-IDF feature representation on text preprocessed with tokenization,

lowercasing, and lemmatization. Our preliminary results establish a strong baseline, with the SVM model achieving a weighed F1-score of 0.58, but our final weighted DeBERTa model and ASR pipeline improved performance, reaching 71.2% accuracy with 71.8% weighted F1-score and ~90% character-level transcription accuracy. This demonstrates the viability of our approach, while also highlighting the difficulty of the task, particularly for underrepresented fallacy classes.

REFERENCES

[1] Z. Jin et al., "Logical Fallacy Detection," arXiv:2202.13758 [cs], May 2022, Available: https://arxiv.org/abs/2202.13758

[2] G. Da, A. Barrón-Cedeño, and Preslav Nakov, "Findings of the NLP4IF-2019 Shared Task on Fine-Grained Propaganda Detection," Archivio istituzionale della ricerca (Alma Mater Studiorum Università di Bologna), vol. D19-50, Jan. 2019, doi: https://doi.org/10.18653/v1/d19-5024.

[3] G. Da San Martino, A. Barrón-Cedeño, H. Wachsmuth, R. Petrov, and P. Nakov, "SemEval-2020 Task 11: Detection of Propaganda Techniques in News Articles," ACLWeb, Dec. 01, 2020. https://aclanthology.org/2020.semeval-1.186/ (accessed Apr. 07, 2022).

[4] Thorsten Joachims, "Text categorization with support vector machines," Research Gate, Oct. 1999, doi: https://doi.org/10.17877/de290r-5097.

[5] Vlad Ermurachi and D. Gîfu, "UAIC1860 at SemEval-2020 Task 11: Detection of Propaganda Techniques in News Articles," International Conference on Computational Linguistics, Dec. 2020, doi: https://doi.org/10.18653/v1/2020.semeval-1.241.

[6] O. Altiti, M. Abdullah, and Ruba Obiedat, "JUST at SemEval-2020 Task 11: Detecting Propaganda Techniques Using BERT Pre-trained Model," International Conference on Computational Linguistics, Dec. 2020, doi: https://doi.org/10.18653/v1/2020.semeval-1.229.

[7] M. Kranzlein, Shabnam Behzad, and Nazli Goharian, "Team DoNotDistribute at SemEval-2020 Task 11: Features, Finetuning, and Data Augmentation in Neural Models for Propaganda Detection in News Articles," International Conference on Computational Linguistics, Dec. 2020, doi: https://doi.org/10.18653/v1/2020.semeval-1.196.

[8] J. Jeong, H. Jang, and H. Park, "Large Language Models Are Better Logical Fallacy Reasoners with Counterargument, Explanation, and Goal-Aware Prompt Formulation," Findings of the Association for Computational Linguistics: NAACL 2022, pp. 6918–6937, Jan. 2025, doi: https://doi.org/10.18653/v1/2025.findings-naacl.384.

[9] A. Lalwani, L. Chopra, C. Hahn, C. Trippel, Z. Jin, and M. Sachan, "NL2FOL: Translating Natural Language to First-Order Logic for Logical Fallacy Detection," arXiv.org, 2024. https://arxiv.org/abs/2405.02318v1 (accessed Oct. 25, 2025).

[10] Z. Sourati et al., "Robust and Explainable Identification of Logical Fallacies in Natural Language Arguments," arXiv.org, 2022. https://arxiv.org/abs/2212.07425 (accessed Oct. 25, 2025).

[11] Y. Lei and R. Huang, "Boosting Logical Fallacy Reasoning in LLMs via Logical Structure Tree," arXiv.org, 2024. https://arxiv.org/abs/2410.12048 (accessed Oct. 25, 2025).